

PROBLEM SET / 05

Temporally grounded graph-assisted discovery

Local graph-assisted discovery with temporal provenance

Boomer Rawlings | Independent individual work

Developed after the original submission and grading dates. Status: In progress.

Abstract

A new search result can reflect a changed world, a late-arriving document, a correction or a different ranking. This GraphRAG discovery application preserves those distinctions through immutable source history, published knowledge snapshots, assertion validity intervals and saved baselines. It runs locally with a Python/SQLite core, a browser workspace and optional llama.cpp extraction and embeddings. [1] [2]

A five-batch fictional pump history exercises plans, late corrections, conflicts and withdrawal. Its authored expectations specify eight historical states and two fixed-knowledge comparisons. A separate actual local-model smoke test extracted two source-grounded assertions and indexed three vectors. These are temporal consistency and integration observations, not a retrieval-quality or scale benchmark. [3] [4] [5]

Problem and design objective

The SCADS problem motivates discovery over connected evidence as a corpus changes. The implemented objective is to make each finding inspectable at a selected knowledge and world-time scope. A relationship must retain its source version and exact passage, while a baseline comparison must survive changes in top-ranked search results. [1] [2] [7]

Implemented contribution

The working system combines lexical, exact dense, hybrid and graph-assisted retrieval with eligibility checks before scoring. It saves investigations and scoped baselines, supports source-linked graph exploration and exports verifiable evidence bundles. Typing previews are a separate bounded keyword path that highlights evidence without calling a model or replacing committed results. [1] [2]

Evidence class: manually authored temporal expectations, actual local-model development smoke tests and fixture/interface verification. Local inference uses no paid API dependency.

1. Research framing and temporal model

Edge et al. introduce GraphRAG for query-focused summarization over large text collections, using graph-derived organization to address corpus-level questions. The present implementation is a different, narrower design: evidence-level lexical/vector seeds, bounded relationship expansion and temporal baseline comparison. It does not implement or reproduce community-summary quality results from that work. [8] [2]

W3C PROV-DM distinguishes entities, activities, derivations, revisions and invalidation. Those distinctions motivate keeping original source versions, extraction activities and later withdrawal separate. The application's contract is project-specific; this correspondence is a conceptual comparison rather than a PROV interoperability claim. [9]

Formal eligibility

Let an assertion a have source revision d , exact span e , modality m , validity interval $[v_start, v_end]$, and publication time k . A query specifies knowledge cutoff K and optional world time t . Eligibility first requires the assertion and relevant source state to be queryable at K ; strict event-time support additionally requires a known interval containing t . Preparation is not publication. [2] [7]

$E(K, t)$ = eligible evidence under knowledge K and world time t

knowledge change: compare $E(K1, t)$ with $E(K2, t)$

world comparison: compare $E(K, t1)$ with $E(K, t2)$

withdrawal: change in support; not proof of a world event

The notation abstracts the implemented event history rather than replacing its full schema. Open-ended validity means no stated end, while unknown bounds require different handling. Planned modality remains planned when a date passes. Comparing rankings alone is insufficient because a relationship can retain support while moving out of top- k results. [2] [3]

Research hypothesis: bounded graph expansion may improve source-supported discovery for relational questions under a fixed resource budget. Temporal correctness is an independent prerequisite, not evidence that the hypothesis is true.

2. Method and implemented architecture

Concern	Implemented representation
Source identity	Immutable revisions, exact UTF-8 hashes and Unicode code-point spans
System knowledge	Separate source availability, receipt, preparation and atomic publication
World claims	Half-open validity intervals, known/open/unknown bounds and reported/planned/negated modality
Retrieval	Lexical scoring, exact cosine, reciprocal-rank fusion and bounded graph expansion
Comparison	Saved findings or fixed neighborhood seeds, paired before/after evidence and scoped coverage

Eligibility is established before vector scoring. An index cannot justify retrieving future evidence into an earlier knowledge view. Immutable vector artifacts bind profile and text hashes; missing eligible vectors fail explicitly. The graph mode expands assertion seeds within declared candidate, hop and evidence limits. [2]

Extraction is a proposal boundary

The local model proposes assertions. The client validates exact quotations and endpoint mentions, creates mention-scoped identities and stages proposals before publication. Unknown dates remain unknown unless explicitly supported. Exact-span validation establishes mechanical grounding, while semantic correctness still depends on what the quoted language actually means. [2] [5]

Three different comparisons

A saved baseline can track selected findings or a fixed entity neighborhood. Knowledge comparisons inspect newly available support, corrections and withdrawals. Fixed-knowledge world comparisons hold the evidence cutoff constant while changing the claimed validity time. These answer different questions: what was learned versus what the sources say happened. [2] [7]

The implementation uses SQLite and exact vectors. Its limits are explicit guardrails rather than benchmarked capacity promises. A displayed neighborhood is a bounded evidence projection, and a net comparison concerns endpoint states in its declared scope; neither implies semantic completeness over the corpus. [2]

3. Retrieval and comparison algorithms

Dense retrieval uses exact cosine on normalized finite vectors. Hybrid retrieval combines dense and positive lexical ranks using reciprocal-rank fusion with constant 60. Stable IDs break score ties. These details are verified in the implementation rather than inferred from a generic GraphRAG description. [10]

```
cosine(q,x) = dot(normalize(q), normalize(x))
RRF(x) = sum over available channels j of 1/(60 + rank_j(x))
channels = dense rank, positive lexical rank
eligibility -> scoring -> bounded seeds -> graph expansion
```

Filtering before scoring prevents future or ineligible evidence from shaping the result pool. Exact query scoring costs $O(Nd)$ vector work over N eligible precomputed vectors of dimension d , before sorting and graph expansion. Index construction additionally includes model embedding generation. Limits on items, dimensions and scalar cells are admission guardrails, not measured throughput. [2] [10]

Baselines are scoped objects

A saved-findings baseline retains selected source lineages and assertions. A neighborhood baseline retains fixed entity seeds and radius. Reconciliation revalidates the baseline scope independently of current top-k ranking. Paired citations explain corrections or lost support; the export preserves before and after evidence rather than writing only an uncited change label. [2] [7]

```
load baseline query, scope and snapshot
project eligible before and after endpoint states
match tracked lineages / explicit supersession / fixed seeds
classify source revision, new support, correction, withdrawal
retain unresolved exclusive-group disputes
export paired original spans, hashes and scope limits
```

Endpoint comparison can miss a change that occurs and reverses between endpoints. It should not be presented as an exhaustive event ledger. This is why the authored demo compares conflict and withdrawal snapshots separately. A future resumable ledger enumeration needs its own completeness and cumulative-budget contract. [2]

4. Worked example

The authored Pump P case uses controlled clocks and explicit one-operator-at-a-time semantics. Every assertion endpoint and interval is stated in its source sentence. The fixture isolates the difference between planned, reported, conflicting and withdrawn support. [3]

Published batch	Evidence meaning	Correct interpretation
January 5	A operates Pump P from January 1	Reported A, with no stated end.
January 20	Plan: A ends and B starts February 1	Passing February 1 does not confirm B.
February 10	Late correction: handover occurred February 3	Availability February 4 differs from receipt/publication February 10.
February 12	Independent report names C from February 3	B and C coexist as disputed claims under fixture exclusivity.
February 15	C source withdrawn	B support survives; withdrawal is not a physical handover.

The same world date, different knowledge

At a February 5 knowledge cutoff, the February 2 world-time view returns B as planned. At a February 11 cutoff, the corrected retrospective February 2 view returns reported A. The question's world date has not changed; the available evidence has. At the February 3 boundary, the later corrected history returns B because validity intervals are half-open. [4]

Compare without future leakage

Holding knowledge fixed at the February 10 publication allows a reported A-to-B comparison between January 31 and February 4. Holding knowledge fixed at the January 20 publication produces a planned transition instead. The later correction must not leak into the earlier answer. [4]

Table 1. All dates and operator claims are fictional test values. The expected-state file was manually specified rather than generated from engine outputs; agreement tests authored semantics, not an actual pump history. [3] [4]

5. Recorded results and interpretation

Recorded observation	Value / result	Interpretation
Local extraction source	73 authored characters	Two simple fictional sentences.
Validated assertions	2: reported and planned	Exact spans; unknown dates retained.
Extraction/publication/query	29.39 s	One local run, excludes model startup.
Vector index	3 vectors x 2,048 components	One source chunk and two assertions.
Vector-phase elapsed	2.44 s	Excludes restart/loading; integration smoke.
Version 0.3.2 checks	138 Python + 30 frontend	Fixture and interface behavior.

The actual local source states that Mira Labs operates Harbor Station and plans to lease Willow Annex. Qwen3-VL-2B-Instruct Q4_K_M through a dedicated llama.cpp server produced two grounded proposals. The queries preserved reported versus planned modality and null temporal bounds. Eight extraction-phase checks and eight vector-phase checks were recorded. [5]

An initial embedding request on the chat-configured server returned HTTP 501; that attempt is preserved. After a separate embedding/pooling configuration, real vectors and query embeddings supported hybrid and graph queries with source-resolving exports. The same decoder weights demonstrate integration only; three exact-index rows do not establish semantic ranking quality or approximate-nearest-neighbor performance. [5]

Interactive evidence without hidden inference

The version 0.3.2 preview debounces typing for 300 ms and uses a separate bounded keyword/prefix endpoint. It creates no model calls or saved runs. Stale responses cannot replace newer scope or committed analysis; Escape, clearing or pausing restores the previous evidence state. Recorded desktop and 390 x 844 mobile checks exercised source tracing and restoration. [2]

The evidence supports a reproducible local system that makes temporal and source distinctions visible. Whether graph expansion improves discovery over lexical or hybrid retrieval remains a separate empirical question; this publication reports no comparative quality uplift. [1] [2]

6. Proposed retrieval and temporal study

PROPOSED STUDY. The design below evaluates quality and scale beyond the small fixture and local-model smoke. No graph-over-baseline improvement is asserted.

Experiment	Control	Primary outcome
Lexical / dense / hybrid / graph	Same corpus, eligible time scope, model and resource budget	Relevant supported findings and ranking
Temporal updates	Independent event/availability/receipt annotations	Future leakage, modality and change classification errors
Extraction	Human-labeled mentions, relations, scope and dates	Full assertion quality, abstention and identity errors
Graph expansion ablation	Seed-only versus bounded hops, matched evidence budget	Incremental relevant discoveries per added cost
Operational scale	Increasing sources, revisions, degree and vector dimensions	Latency, memory, indexing/update cost and completeness

Freeze questions by relation type and difficulty, separating local fact lookup, multi-hop dependency and baseline-change tasks. Split related document families and revisions together so that near duplicates do not leak labels across development and test. Annotators should judge source-supported relevance, temporal eligibility and identity separately. [11]

Report precision/recall of supported discoveries within declared candidate scope, citation-span correctness and the rate of unsupported temporal transitions. Assess rank quality only on judgments with adequate coverage; unknown relevance cannot be silently treated as a negative. Pair query-level differences across retrieval variants and include confidence intervals clustered by source family where appropriate.

Measure query and ingestion costs together. Graph construction can move work from query time to preprocessing, while updated histories require invalidation and reindexing. Compare systems under a fixed measured budget and report failures or truncation. A negative graph result remains informative if the strongest non-graph baseline and resource scope are documented. [1] [11]

7. Roadmap and validity of discovery

Phase	Output	Progression criterion
Qualify corpus and time	Versioned public sources and independent temporal/relevance labels	Separate world events from publication and receipt.
Complete comparison semantics	Resumable event ledger and cumulative resource accounting	Demonstrate declared scope completeness or explicit truncation.
Resolve and review identities	Versioned reversible identity/assertion decisions	Retain originals and prevent silent historical rewriting.
Evaluate retrieval and scale	Paired baselines, temporal audit and measured cost curves	Publish useful positive or negative results within scope.

The existing application is In progress. These phases extend its inspectable local reference implementation rather than promise a medium- or large-scale outcome. Approximate indexes or another database should follow a measured bottleneck and be evaluated against the exact reference, including update and cutoff behavior. [2]

Threats to validity

The pump fixture assumes explicit exclusivity and exact entity names. Real contradictions require more than similar relation text, and mention-scoped IDs can fragment repeated entities. Exact quotation prevents fabricated spans but not misinterpreted relations. The local smoke uses simple sentences and a decoder model for embedding plumbing, so it cannot estimate extraction or semantic retrieval quality. [3] [5]

Historical replay also requires care: a later vector artifact over retained source text can be cutoff-safe without reproducing the model or index that actually existed at that historical time. Model identity, pooling, source versions and index fingerprints must therefore accompany a comparison. Display animations illustrate returned support; they do not reveal a model's internal reasoning. [2] [5]

Conclusion

The implementation demonstrates temporally scoped discovery with inspectable source history, explicit plans and corrections, saved baselines and local inference integration. Its research contribution is an executable framework for asking whether graph assistance adds useful discovery under measured constraints. The answer must come from the controlled evaluation rather than the visual appeal of a graph.

8. Reproducibility and research record

The authored demo needs only the application runtime and creates a new database, report and evidence bundles. Use fresh paths: the demo refuses to overwrite existing output. Local extraction and embeddings require the separately configured llama.cpp model service documented in the source. [1] [2] [6]

```
python -m graphrag_discovery --db runs/demo.sqlite3 demo --output runs/demo
python -m graphrag_discovery --db runs/demo.sqlite3 serve --port 8765
python -m unittest discover -s tests -v
```

Run from the nested project directory or an installed environment. The service binds to loopback; the authored demo uses simulated time and does not make model requests. Wheel/sdist installation instructions distinguish the application from separately installed model assets. Evidence bundles preserve source spans and hashes for inspection. [2] [6]

The static publication does not execute new model or performance experiments. Recorded development observations, authored temporal semantics and measured software behavior remain explicitly separate.

A reproducible research record

The public nested project source contains implementation, documented contracts and retained development evidence. The publication evidence manifest records SHA-256 fingerprints for each local source cited in this paper. External references link to primary publications or official technical documentation, verified for this edition.

A future rerun should record the exact commit, dependency/runtime identities, input hashes, configuration and environment, plus each attempt and its outcome. Keep generated outputs separate from source and mark any changed implementation as a new condition. Reproduction of a software check does not automatically reproduce a model judgment, benchmark distribution or independent human label.

Authorship and project status

Boomer Rawlings developed this work individually after the original submission and grading dates. The current project is In progress. The implemented methods and recorded observations in this paper are separated from the explicitly proposed research protocol and roadmap. No group-project, graded-submission or completed-research claim is made.

References

Repository references identify the evidence used for the implementation and result claims. Primary literature supports the research framing; its published results are not attributed to this project.

[1] [Current application and execution policy](#)
IMPLEMENTATION.md

[2] [Implemented retrieval, history and interface](#)
docs/implementation-status.md

[3] [Authored temporal pump fixture](#)
fixtures/pump/README.md

[4] [Expected temporal states and comparisons](#)
fixtures/pump/gold.json

[5] [Actual local-model and vector smoke test](#)
docs/local-model-validation.md

[6] [Installation and package reproduction](#)
docs/installation.md

[7] [Temporal design and comparison semantics](#)
docs/temporal-workflow.md

[8] [Edge et al. \(2024; revised 2025\). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130.](#)
<https://arxiv.org/abs/2404.16130>

[9] [Moreau and Missier, eds. \(2013\). PROV-DM: The PROV Data Model. W3C Recommendation.](#)
<https://www.w3.org/TR/prov-dm/>

[10] [Implemented exact vectors and rank fusion](#)
graphrag_discovery/vectors.py

[11] [Research comparison and resource evaluation design](#)
research/data-and-evaluation.md